

PROGRAMMAZIONE II

CLASSI ASTRATTE E INTERFACCE

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

CLASSI ASTRATTE



METODO ASTRATTO

Un metodo astratto è un metodo di classe che non ha codice

- ▶ Descrive una funzionalità fornita dalla classe
- ▶ Ma non fornisce un'implementazione per tale funzionalità

I metodi astratti sono definiti usando la parola chiave `abstract`

modificatori `abstract` *tipo nomeMetodo (parametri) ;*

Per esempio: `public abstract int computeOnAandB(int paramA, int paramB);`

CLASSE ASTRATTA

Una classe astratta è una classe che contiene almeno un metodo astratto

- ▶ Informalmente, è una classe incompleta
- ▶ Quindi, non può essere istanziata

```
public abstract class Shape { // se c'è un metodo astratto la      1
    private int color;        // classe deve essere astratta      2
    public void setColor(int color) { this.color = color; }        3

    // il comportamento di draw non è specificato                  4
    public abstract void draw();                                    5
}                                                                    6
                                                                    7
```

Object obj = new Shape() genererà un errore di compilazione

CLASSE ASTRATTA (CONT.)

Una classe astratta richiede una sottoclasse che implementi i suoi metodi astratti

- ▶ Può essere una classe concreta o un'altra classe astratta
- ▶ Se non astratta, può essere istanziata

// tutti i metodi implementati

```
public class Circle extends Shape {           1
                                                2
    // il comportamento di draw è specificato solo per questa sottoclasse  3
    public void draw() {                       4
        // codice per disegnare un cerchio    5
    }                                           6
}                                              7
```

Object obj = new Circle(); non solleverà errori

MOTIVAZIONE DELLE CLASSI ASTRATTE

Una superclasse può essere usata per definire il comportamento comune a più classi figlie

- ▶ Ma la classe potrebbe essere troppo generica per essere istanziata
- ▶ Il suo comportamento deve essere parzialmente non specificato (specifico per la figlia)

 Una classe astratta può estendere una classe concreta

UN ORDINATORE PER MOLTI TIPI

```
1 public abstract class ArraySorter {
2     // l'algoritmo di ordinamento dovrebbe essere lo stesso per tutti i tipi di array
3     public final void sort(Object[] arr) {
4         for (int i = 1; i < arr.length; i++)
5             for(int j = 1; j < arr.length; j++)
6                 if (greaterEqual(arr[j-1], arr[j])) {
7                     Object obj = arr[j];
8                     arr[j] = arr[j-1];
9                     arr[j-1] = obj;
10                }
11
12     // ma il controllo del confronto dipende dal tipo degli elementi dell'array
13     protected abstract boolean greaterEqual(Object a, Object b);
14 }
```

UN ORDINATORE PER MOLTI TIPI (CONT.)

```
public class IntArraySorter extends ArraySorter {           1
    // questa implementazione è specifica per gli array di int 2
    @Override                                                3
    protected boolean greaterEqual(Object a, Object b) {    4
        Integer ia = (Integer) a; // qui avremmo dovuto controllare il cast! 5
        Integer ib = (Integer) b; // qui avremmo dovuto controllare il cast! 6
        return ia >= ib;                                     7
    }                                                         8
}                                                            9
```

IntArraySorter funziona per Integer[] ma non per String[]

- Dobbiamo implementare una classe concreta anche per i valori String

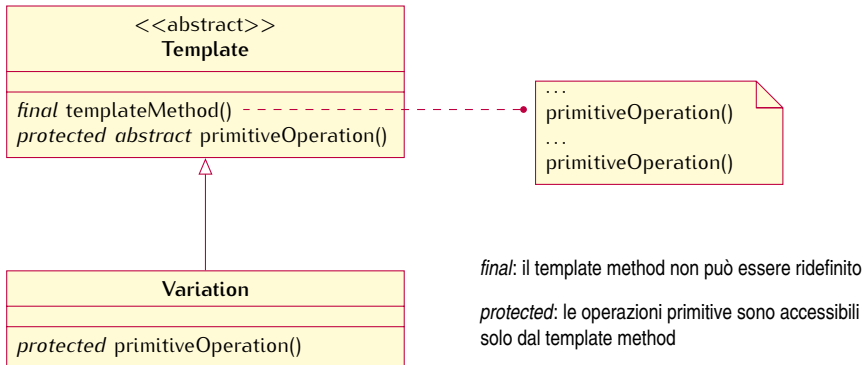
PATTERN TEMPLATE METHOD

Contesto: Un comportamento ha un nucleo stabile e diverse variazioni

Problema: Diversi frammenti di codice simili devono essere implementati e mantenuti

Soluzione: Separare il nucleo dalle variazioni (template method)

PATTERN TEMPLATE METHOD (CONT.)



INTERFACCE



INTERFACCIA

Un'interfaccia è una classe che contiene solo metodi astratti

- ▶ Quindi, non può essere istanziata
- ▶ Modella il comportamento di un componente, disaccoppiandone l'implementazione

Un'interfaccia è un tipo?

- ▶ Non per gli oggetti (nessun oggetto può essere istanziato come interfaccia)
- ▶ Ma è un tipo per i riferimenti

 Lo stesso vale per le classi astratte

INTERFACCIA (CONT.)

In un'interfaccia tutti i metodi e i campi sono implicitamente **public**

- ▶ Anche i metodi sono implicitamente **abstract**
- ▶ Anche i campi sono implicitamente **static** e **final** (costanti)

```
interface MyInterface {  
    int A_FIELD = 0;           // public static final  
    void aMethod(String aParam); // public abstract  
    ...  
}
```

1
2
3
4
5

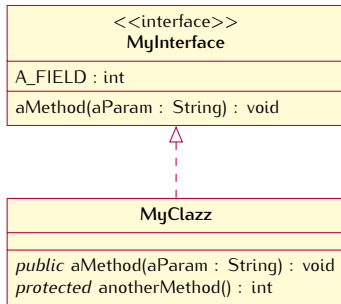
IMPLEMENTARE UN'INTERFACCIA

Quando una classe implementa un'interfaccia

- ▶ Deve fornire un'implementazione per tutti i metodi dell'interfaccia
- ▶ Può definire metodi aggiuntivi

```
class MyClass implements MyInterface { 1
    public void aMethod(String aParam) { // qui dev'essere indicato public 2
        if (A_FIELD != Integer.parseInt(aParam)) 3
            System.out.println(aParam); 4
    } 5
    protected int anotherMethod() { return 42; } 6
} 7
```

DI NUOVO IL PRINCIPIO DI SOSTITUZIONE



MyClazz è una sottoclasse di MyInterface

► `MyInterface ref = new MyClazz();`

```

ref.aMethod("str");           // ok           1
ref.anotherMethod();          // errore di compilazione 2
((MyClazz) ref).anotherMethod(); // ok           3
    
```

INTERFACCE ED EREDITARIETÀ

Un'interfaccia può estendere un'altra interfaccia, ma non può estendere una classe

```
interface MyInterface extends Comparable { ... }
```

In realtà, un'interfaccia può estendere più interfacce

```
interface MyInterface extends Comparable, Orderable { ... }
```

Una classe può estendere una sola classe, ma può implementare più interfacce

```
class Student extends Person implements Comparable { ... }
```


MOTIVAZIONE DELLE INTERFACCE

Le interfacce forniscono un completo disaccoppiamento tra i messaggi che un oggetto può ricevere e la loro implementazione

- ▶ Nessun codice condiviso, diversamente dalle classi astratte
- ▶ Implementazioni alternative

```
public interface ComplexNumber {  
    double read();  
    double imaginary();  
    double modulus();  
    double argument();  
} // può essere implementata usando coordinate cartesiane o polari
```

1
2
3
4
5

INTERFACCIA COMPARABLE

L'interfaccia `java.lang.Comparable` contiene il metodo

// semplificata

```
int compareTo(Object obj);
```

Con la seguente semantica:

- ▶ Restituisce un intero **negativo** se **this** è minore di `obj`
- ▶ Restituisce **zero** se **this** è uguale a `obj`
- ▶ Restituisce un intero **positivo** se **this** è maggiore di `obj`

Usata nella classe `java.util.Arrays` per ordinare gli array

INTERFACCIA COMPARABLE (CONT.)

```
1 public class Student extends Person implements Comparable {
2     ...
3     @Override
4     public int compareTo(Object obj) {
5         Student other = (Student) obj; // dovremmo controllare il cast
6         return this.matricola - other.matricola;
7     }
8 }
9
10 Student[] arrS = {new Student("Sam", 456), new Student("Paul", 123)};
11 Person[] arrP = {new Person("Sam"), new Person("Paul")};
12
13 Arrays.sort(arrS); // ok --array ordinato
14 Arrays.sort(arrP); // errore a runtime --ClassCastException
15                     // Person non può essere castato a java.lang.Comparable
```

INTERFACCE ITERABLE E ITERATOR

L'interfaccia `java.lang.Iterable` contiene il metodo

// semplificata

```
Iterator iterator();
```

Una classe che implementa `Iterable` può essere usata nei cicli enhanced for

- ▶ L'interfaccia usa l'interfaccia `Iterator`
- ▶ Per iterare su una collezione di elementi

INTERFACCE ITERABLE E ITERATOR (CONT.)

L'interfaccia `java.util.Iterator` contiene i metodi

// semplificata

```
boolean hasNext(); e Object next();
```

Con la seguente semantica:

- ▶ Inizia l'iterazione prima del primo elemento
- ▶ `hasNext()` indica se è presente un elemento successivo
- ▶ `next()` restituisce l'elemento successivo e avanza di una posizione

INTERFACCE ITERABLE E ITERATOR (CONT.)

```
import java.util.Iterator;
import java.lang.Iterable;

class ClassRoom implements Iterable {
    private Student[] students;

    ClassRoom(Student[] students) {
        this.students = students;
    }
    @Override
    public Iterator iterator() {
        return new CRIterator(students);
    }
} // avremmo potuto implementare questa
// classe in modo più elegante usando
// una classe interna
```

```
import java.util.Iterator;

class CRIterator implements Iterator {
    private Student[] arr;
    private int next = 0;

    CRIterator(Student[] arr) { this.arr = arr; }
    @Override
    public boolean hasNext() {
        return next < arr.length;
    }
    @Override
    public Object next() {
        return new Student(arr[next++]);
    }
}
```

```
Student[] arrS = {new Student("Sam", 456), new Student("Paul", 123)};
ClassRoom classRoom = new ClassRoom(arrS);
for (Object obj : classRoom) // per rendere il codice più elegante servono i generici
    System.out.println(((Student) obj).toString());
```



DOMANDE E RISPOSTE

